

G06 / PROTECTED ANSWERS

G06 REVIEW A

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

Case reminder 3

One defensible solution shape 3

Implementation/runtime rubric 3

Concrete requirement-to-decision mapping 3

Pass / repair decision 4

Changed recovery example 4

Resolve the latency ambiguity first 4

Capacity calculation, including an hourly burst 5

Explicit recovery targets and a failure test 5

Worked monthly planning budget 5

A concrete least-privilege matrix 5

Promotion, rollback and ownership 6

Execute two supplied local demonstrations 6

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open after the matching saved Gate attempt.

Case reminder

You are reviewing a cloud data platform for a regional insurer. Claims arrive as hourly files (25 GB/day). Fraud rules can wait 15 minutes, executive reporting can wait 2 hours. Accepted raw evidence must never be lost. PII/health data requires restricted access. The data team has five engineers and no dedicated Kubernetes platform team. Current cost target is 18,000 units/month. The old design copies full raw data into three systems and uses one shared admin service account.

One defensible solution shape

A strong solution begins with requirements and separates real-time risk decisions from slower governed reporting. Preserve immutable accepted source evidence, make replay/checkpoints explicit, use managed components unless custom ownership is justified, keep sensitive identities isolated by least privilege, publish only after reconciliation/quality checks, and retain last-known-good serving state on failed validation.

Implementation/runtime rubric

- Requirement trace: every major component has a stated requirement/constraint.
- Reliability: RTO/RPO, failure domains and replay are explicit; no checkpoint outruns durable output.
- Security: distinct personas/identities; no wildcard owner shortcut; sensitive data boundary is clear.
- Scale: one numerical throughput/headroom check and bottleneck response.
- Cost: current and sensitivity case; optimization does not delete evidence/recovery.
- Promotion: CDC/contracts/Vault/Mesh/Kubernetes are included only where they solve a scenario problem.
- Runtime evidence: at least two local scripts/scorecards run; one deliberate failure and recovery; one changed-input
- retry.
- Critical fail if the answer claims a cloud/engine run that was not actually executed.

Concrete requirement-to-decision mapping

Latency split: define whether the fraud SLA begins at source-event time or at file arrival. scheduled/micro-batch ingestion with immutable raw claims; fraud scoring may run every 5-15 minutes while executive serving refreshes within the 2-hour window.

Platform ownership: with five data engineers and no Kubernetes platform team, prefer managed orchestration/compute/warehouse-lakehouse services unless a hard control requirement says otherwise. Self-hosting Kafka/Kubernetes is a weak default here.

Reliability: RPO zero for accepted raw evidence means never advance progress before durable landing. Keep replayable immutable raw data, versioned/controlled curated publication and last-known-good reporting. Test a failed validation and prove the old serving version remains available.

Security: separate ingest, transform, fraud analyst, reporting and restricted-health-data personas. Replace the shared admin account with least-privilege workload identities; mask/tokenize sensitive fields before general BI access.

Cost: eliminate three full raw copies before cutting recovery controls. Track storage/compute/egress separately and compare managed service premium against on-call/upgrade labor. Stay under the monthly target with a sensitivity case.

Pass / repair decision

- PASS: architecture traces each major choice to a requirement and includes runnable local evidence plus a
- deliberate failure/recovery and changed retry.
- REPAIR: design is plausible but lacks one critical control such as replay, least privilege, capacity math, or cost
- sensitivity. Repair only that weak area, then retest with changed input.
- FAIL: broad admin identity, progress ahead of durable output, no saved first attempt, no recovery path, or invented
- claims of cloud/runtime execution.

Changed recovery example

If the first architecture chooses streaming everywhere, challenge it by separating fast decisions from slower governed reporting. If it self-hosts Kafka/Kubernetes without a platform reason, recalculate operational ownership. If it centralizes all PII, redraw trust boundaries. If cost exceeds the target, reduce scans/copies or right-size compute before deleting resilience.

Resolve the latency ambiguity first

Hourly source files cannot guarantee fraud decisions within 15 minutes of the underlying claim event. A five-minute scheduler cannot process a record that has not arrived. Ask whether the 15-minute target begins at file arrival or at claim creation. For this reference, assume it starts at durable file arrival. If it starts at claim creation, require a faster upstream event/CDC feed or renegotiate the SLA before approving the architecture. Mark this as an unresolved requirement until the owner answers.

Capacity calculation, including an hourly burst

Use decimal GB/MB units for this exercise. 25 GB/day is about 1.04 GB per hourly delivery and 0.289 MB/s averaged over a day. The average hides the hourly burst. To process 1.04 GB in ten minutes requires roughly 1.74 MB/s, leaving five minutes for arrival detection, checks and publication within a 15-minute post-arrival target. At 2× daily volume the corresponding processing requirement is 3.47 MB/s. These are minimum data-throughput estimates before parsing, shuffle, skew, concurrency and output overhead; measure those on representative files.

One capacity hypothesis is that transformation or sink throughput is the bottleneck, not ingestion scheduling. Test it by measuring bytes processed and elapsed time at each stage. A stage that sustains only 2 MB/s meets the current simplified ten-minute window but misses the doubled one. Reducing scan/shuffle volume or improving the measured bottleneck is more useful than changing every component.

Explicit recovery targets and a failure test

Assume an operational target RTO of 30 minutes for curated reporting; the business must approve it. No RTO was supplied by the question. RPO zero for acknowledged raw evidence means acknowledging only after durable storage succeeds, with failure-domain replication appropriate to the declared risk. It is not achieved merely by keeping two files on one disk. Raw retention and access protection are separate from curated-table checkpoints.

Inject a failure after curated output is prepared but before it is published. The last validated report must remain readable; progress must still refer to the last completed input. Replay the same raw identity, validate counts and totals, then publish once. Record the last durable state and the measured recovery time. A local simulation proves the sequence, not the availability of a managed cloud service.

Worked monthly planning budget

These are hypothetical units from the case, not current vendor prices: durable storage/recovery 2500; ingestion 1000; transformations 6000; orchestration/monitoring 1500; serving 3000; networking 1000; contingency 3000. Total 18,000. At 2× data, suppose storage becomes 5000, ingestion 2000, transformation 9000, monitoring 1800, serving 3600, networking 1500 and contingency 3000: total 25,900. The old budget would then be exceeded by 7900. Incremental processing and eliminating redundant non-recovery raw copies are candidates to measure; do not delete the only recovery copy to meet a budget.

A concrete least-privilege matrix

Ingest identity: write new raw deliveries, no general reporting access and no delete permission. Transform identity: read its assigned raw zone and write versioned trusted outputs, not administer all identities. Fraud workload: read only required claim/order fields and its restricted identity lookup. BI users: read approved marts with direct identifiers removed or access-limited. Operations: read logs and perform authorized recovery without routine access to raw health/identity content. Human emergency administration: exceptional, time-limited and audited, not a shared service login.

Promotion, rollback and ownership

Use reviewed code and pinned environment configuration; run unit/contract/integration checks, validate a representative canary partition and promote only after its business controls pass. Keep the previous deployable code and output generation. Roll back code when the regression is code-related; repair and replay affected data from raw evidence when a bad transformation was already published. An older code deployment does not automatically undo wrong data.

Contracts are justified now because producer/consumer fields and business identities need explicit rules. CDC is justified only when a source database and freshness/change-capture requirement demand it. A managed secret store/workload identity is justified; self-hosted Vault is not automatically necessary. Data mesh is an ownership model, not a tool to install for a small team. Kubernetes requires an actual platform/control requirement and operating capacity; an eight-person platform team may make it feasible in B but does not make it compulsory.

Execute two supplied local demonstrations

From the course root, run these standard-library examples:

```
python 15_M16/practice/dataops_patterns/03_quality_gate.py
python 16_M17/practice/architecture_patterns/05_bottleneck_capacity_reasoning.py
```

The quality example contains duplicate id=2 and prints unique_id=false with decision QUARANTINE. Its assertion confirms the intended failure demonstration, so the program can exit successfully while the business data fails. Do not interpret exit code zero as publish permission. In a review copy, remove the duplicate or change it to a legitimate distinct row, update the demonstration assertion to expect PASS, and run again. Then introduce a different duplicate as a changed retry and expect QUARANTINE. Record all three business decisions.

The capacity example prints sink as the bottleneck: capacity 12000, target 20000; ingest=50000 and spark=30000. To model a corrected target of 10000, copy the script and update both target and the expected comparison assertion deliberately; unchanged assertions are tied to the original teaching case. To model a higher-capacity sink, keep the required target and recompute which stage is now smallest. These are scorecard/simulator checks. They do not execute Kafka, Spark or a cloud deployment.